Rockchip IMU Developer Guide

ID: RK-KF-YF-H58

Release Version: V1.0.0

Release Date: 2026-04-29

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document introduces the configuration methods and basic debugging methods for IMU (Inertial Measurement Unit) drivers on the Rockchip Linux platform.

Product Version

Chipset	Kernel Version
ALL	Linux 6.1 / Linux 5.10

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Reinhardt Zhang	2026-04-29	Initial version

Contents

Rockchip IMU Developer Guide

1. Linux IIO Framework
    1.1 Linux IIO Framework Overview
        1.1.1 IMU System Hierarchy
        1.1.2 Driver Layer Responsibilities
    1.2 IIO Subsystem Key Components
        1.2.1 IIO Device
            1.2.1.1 IIO Device Channel
            1.2.1.2 IIO Device Attributes
        1.2.2 IIO Buffer
            1.2.2.1 Buffer Configuration
            1.2.2.2 Scan Element Configuration
        1.2.3 IIO Trigger
            1.2.3.1 Trigger Callback
        1.2.4 Triggered-Buffer
    1.3 Driver Adaptation Process for IIO Framework
        1.3.1 DTS & Kconfig
            1.3.1.1 Bus Configuration
            1.3.1.2 Interrupt / Pin Configuration
            1.3.1.3 Kconfig Configuration
        1.3.2 Driver Framework Setup
        1.3.3 Define IIO Device Channels and Attributes
        1.3.4 Chip Initialization Implementation
        1.3.5 Configure IIO Trigger
        1.3.6 Configure Triggered-Buffer
2. IMU User Guide
    2.1 IIO sysfs Configuration
        2.1.1 Sampling Frequency
        2.1.2 Range and Precision
        2.1.3 Low-Pass Filter
        2.1.4 Interrupt Enable
    2.2 Data Reading and Processing
        2.2.1 IIO sysfs Read
        2.2.2 IIO Buffer Read Process
            2.2.2.1 Confirm Device Node Path
            2.2.2.2 Understand Frame Layout
            2.2.2.3 Read Scale Value
            2.2.2.4 Enable Buffer
            2.2.2.5 Loop Read Frames
            2.2.2.6 Convert and Print Data
3. Data Analysis and Calibration
    3.1 Data Calibration and Calibration
        3.1.1 Understand the Calibration Parameters
        3.1.2 Install the Calibration Tool
        3.1.3 Collect Calibration Data
        3.1.4 Create Configuration Files
        3.1.5 Run Calibration
        3.1.6 Write Calibration Results into the Algorithm Configuration
    3.2 Determining Data Normalcy
        3.2.1 Data Acquisition
        3.2.2 Data Check

4. FAQ
    4.1 PCB Layout Precautions
    4.2 Is Zero Bias and Temperature Drift Factory Calibration Needed
    4.3 Effect of Built-in LPF on Data

1. Linux IIO Framework

This chapter introduces the overall architecture and key components of the Linux IIO (Industrial I/O) subsystem, and provides a complete process for adapting an IMU chip to the IIO framework. The content in this chapter is independent of specific IMU chip models and applies to all IMU driver development based on the IIO framework.

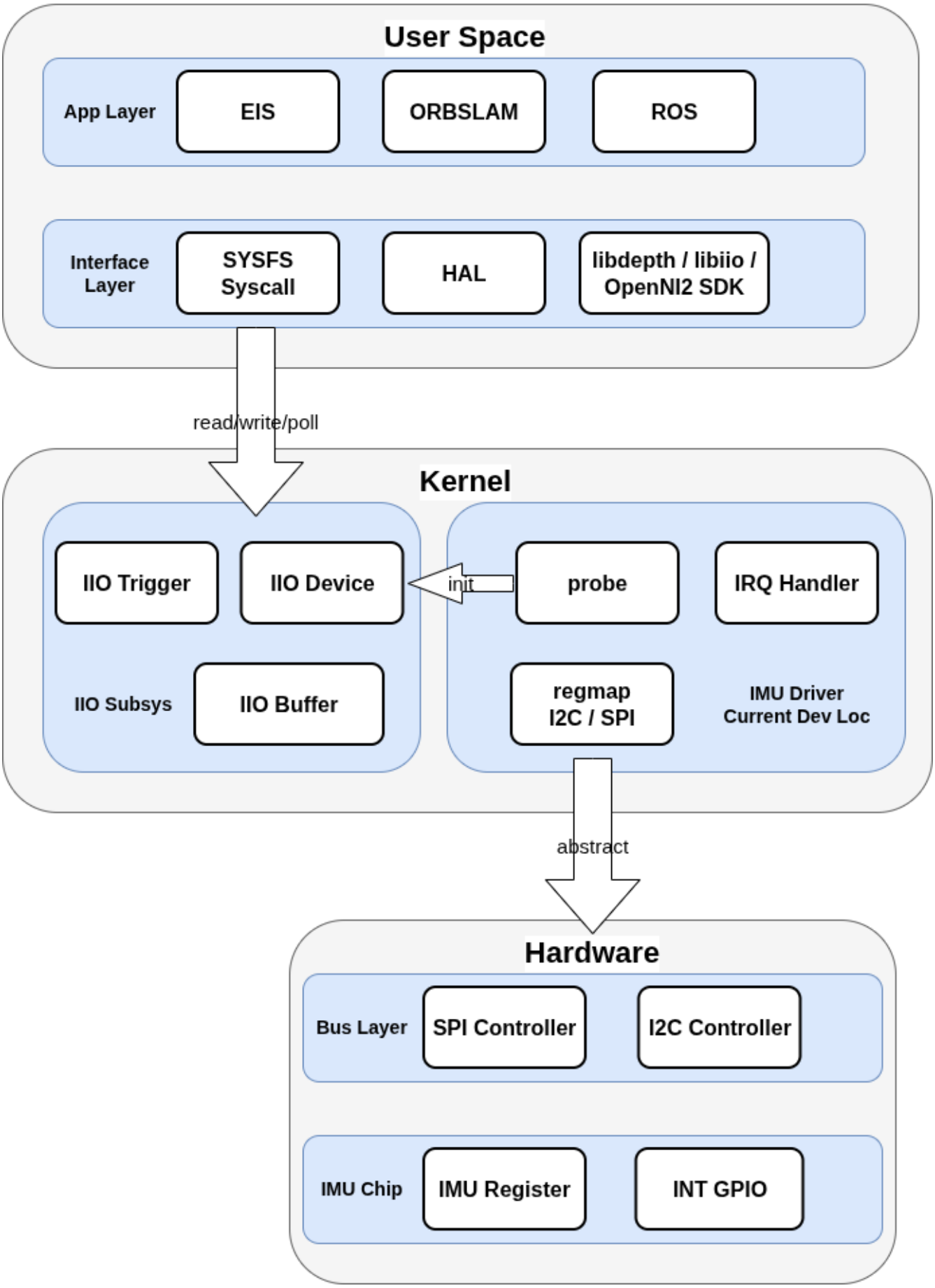
1.1 Linux IIO Framework Overview

IIO is a Linux kernel subsystem specifically designed for analog/mixed-signal devices such as ADC, DAC, accelerometers, and gyroscopes. The IIO subsystem provides a unified framework for writing drivers for various embedded sensors, as well as a standard interface for user-space applications to control these sensors.

1.1.1 IMU System Hierarchy

The figure below shows the complete hierarchy of the IMU software stack.

IMU Software Stack



1.1.2 Driver Layer Responsibilities

The driver layer is responsible for **registering devices with the IIO framework, adapting the underlying communication bus, and configuring triggers and buffers**.

1.2 IIO Subsystem Key Components

This section describes the role of each key component in the IIO subsystem and the driver-side implementation.

Component	Structure	Header File	Function
IIO Device	`iio_dev`	`linux/iio/iio.h`	Represents an IIO device
IIO Device Channel	`iio_chan_spec`	`linux/iio/iio.h`	Describes a single sensor channel
IIO Device Attribute	`iio_info`	`linux/iio/iio.h`	IIO attribute and callback collection
IIO Trigger	`iio_trigger`	`linux/iio/trigger.h`	Sampling trigger source in sensors
IIO Buffer	`iio_buffer`	`linux/iio/buffer.h`	Stores sensor sampling data

1.2.1 IIO Device

An IIO device is a software abstraction of a hardware sensor. Upper-layer users control sensors by accessing the IIO device's sysfs files or character device files. Driver developers need to allocate an IIO device via `iio_device_alloc` / `devm_iio_device_alloc`, mount IIO device channels and attributes onto the IIO device, and finally register it with the IIO framework via `iio_device_register` / `devm_iio_device_register`.

1.2.1.1 IIO Device Channel

An IIO device channel is an abstract representation of a data channel. An IIO device can have one or more channels. For example, an accelerometer can have up to three channels representing acceleration on the X, Y, and Z axes respectively. IIO channels are described by the `iio_chan_spec` structure. Taking an accelerometer as an example, the driver program needs to define it as follows:

```
static const struct iio_chan_spec accel_x_channel[] = {
    {
        .type = IIO_ACCEL,            // Accelerometer type
        .channel2 = IIO_MOD_X,        // X-axis modifier
        .modified = 1,
        .info_mask_separate = BIT(IIO_CHAN_INFO_RAW),
        .info_mask_shared_by_type = BIT(IIO_CHAN_INFO_SCALE) |
BIT(IIO_CHAN_INFO_SAMP_FREQ),
    },
};
```

When the Linux kernel starts, it creates channel private files `in_accel_x_raw` and shared files `in_accel_scale` & `in_accel_sampling_frequency` in the `/sys/bus/iio/devices/iio:deviceX/` directory. The sysfs file naming format is:

```
in_{chan_type}{channel && indexed}_{channel2 && modified}_{info_type}
```

1.2.1.2 IIO Device Attributes

`iio_info` defines constant attributes and information for IIO devices. Only the attributes highly relevant to IMU driver development are listed here.

attrs

attrs defines some common attributes of IIO devices. For example, the optional range and sampling frequency parameters of accelerometers and gyroscopes, as well as general sensor filter configuration, can be implemented through attrs attributes.

```
// Configure read-only attribute
static IIO_CONST_ATTR(in_accel_sampling_frequency_available,
                      "1 10 100 1000 10000 100000");

static struct attribute *xxx_attrs[] = {
    &iio_const_attr_in_accel_sampling_frequency_available.dev_attr.attr,
};
```

read_raw

read_raw defines the method for users to read IIO data channel sysfs files.

```
int (*read_raw)(struct iio_dev *indio_dev,
                struct iio_chan_spec const *chan,
                int *val, int *val2, long mask);
```

Parameter Description

- chan - Data type, such as IIO_ACCEL / IIO_ANGL_VEL / IIO_TEMP
- mask - sysfs file attribute, such as IIO_CHAN_INFO_RAW / IIO_CHAN_INFO_SCALE / IIO_CHAN_INFO_SAMP_FREQ

- val - Integer reading
- val2 - Fractional reading

The `read_raw` return value and data printing format are as follows:

Return Value	Data Printing Format
IIO_VAL_INT	val
IIO_VAL_INT_PLUS_MICRO	$val + val2 \times 10^{-6}$
IIO_VAL_INT_PLUS_NANO	$val + val2 \times 10^{-9}$

write_raw

write_raw defines the method for users to write IIO data channel sysfs files.

```
int (*write_raw)(struct iio_dev *indio_dev,
                 struct iio_chan_spec const *chan,
                 int val, int val2, long mask);
```

It is recommended to add validation via `iio_device_claim_direct_mode` before reading/writing to prevent data errors caused by concurrent register modifications during buffer operation.

1.2.2 IIO Buffer

The IIO framework provides a buffer as a temporary storage space for sensor data, with the underlying implementation based on `kfifo` circular buffer. Upper-layer users can read one or more frames of data at a time from the `/dev/iio:deviceX` character device node, thereby reducing CPU load. The driver itself does not need to manage buffer memory.

1.2.2.1 Buffer Configuration

The driver creates a buffer directory in sysfs via the `devm_iio_triggered_buffer_setup` interface, providing the following configurable key parameters.

- `buffer/length`: Configures the number of records the buffer can hold. The actual memory usage is approximately $length × single record length$. Default value is 2.
- `buffer/watermark`: Configures the watermark for the number of sampling records after which the kernel sends a readable event to user space. Default value is 1.
- `buffer/enable`: Enables IIO buffer acquisition. Write 1 to enable, write 0 to disable. After enabling, the IIO framework automatically callbacks `set_trigger_state` to enable the trigger.

1.2.2.2 Scan Element Configuration

`scan element` is the metadata associated with channel readings stored in the `IIO buffer`. IIO channels define how data channels are written to the IIO buffer. Still taking the previous accelerometer as an example, the IIO channel definition is as follows:

```
static const struct iio_chan_spec accel_x_channel[] = {
    {
        .scan_index = 0,            // Write buffer arrangement order
        .scan_type = {
            .sign = 's',            // Sign bit
            .realbits = 16,         // Actual data bits
            .storagebits = 16,      // Storage bits, must be 8-bit aligned, pad if not
aligned
            .endianness = IIO_BE,   // Storage byte order; configure big-endian: high-order
byte at low address
        },
    },
};
```

After channel mounting, during kernel initialization, `scan_element` configuration files are created in the sysfs directory `/sys/bus/iio/devices/iio:deviceX/scan_elements/`.

- `scan_elements/in_*_en`: Used to enable the channel. Only when its attribute value is non-zero will a triggered acquisition include this channel's data sample.
- `scan_elements/in_*_index`: Determined by `scan_index`, the acquisition write buffer arrangement order, sorted by number from smallest to largest.
- `scan_elements/in_*_type`: The storage format of data in the buffer, with the format: `[be|le]:[s|u]bits/storagebits[Xrepeat][>>shift]`

1.2.3 IIO Trigger

The IIO framework provides a trigger mechanism that allows users to control the enabling/disabling of hardware interrupts. The driver allocates a trigger via `devm_iio_trigger_alloc`, defines callbacks in the `trigger->ops` field, and registers it with the IIO framework via `devm_iio_trigger_register`. During driver initialization, the following files are created in the sysfs directory:

- `/sys/bus/iio/devices/triggerY/*`: After successful trigger registration, the kernel automatically assigns it a number Y and creates attribute files in this directory. The trigger name can be queried via the `name` attribute.
- `/sys/bus/iio/devices/iio:deviceX/trigger/*`: Created after enabling the triggered buffer via `devm_iio_triggered_buffer_setup`. Provides the configuration file `current_trigger` to change the current trigger name.

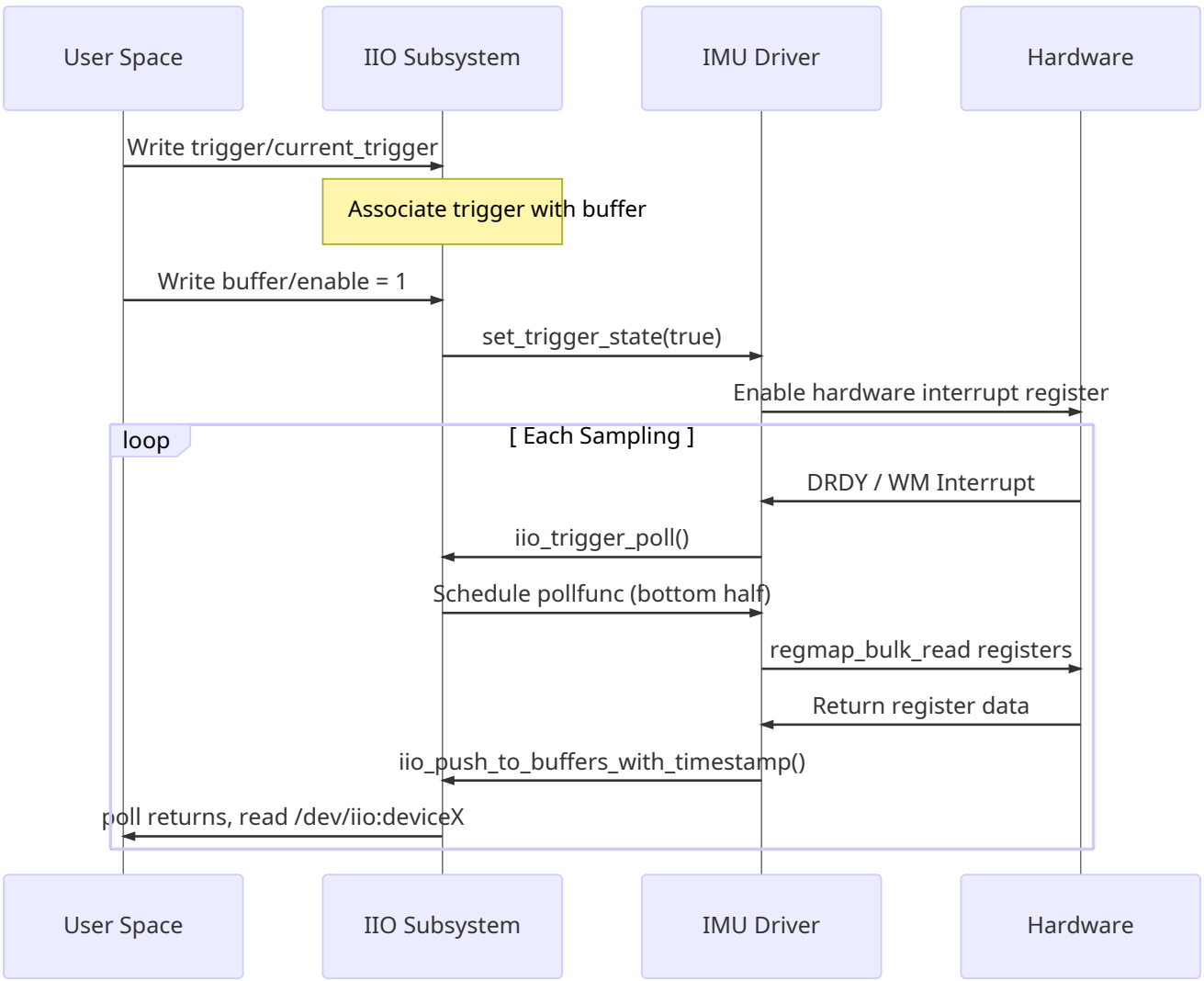
1.2.3.1 Trigger Callback

The `set_trigger_state` callback is responsible for enabling or disabling the trigger. When user space enables `buffer/enable` through sysfs (see Interrupt Enable), the IIO framework executes this callback function.

```
struct iio_trigger_ops {
    // Set trigger state
    int (*set_trigger_state)(struct iio_trigger *trig, bool state);
};
```

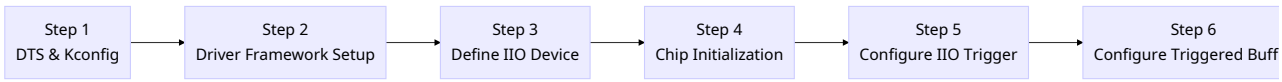
1.2.4 Triggered-Buffer

As can be seen from the preceding text, IIO triggers and buffers need to work closely together. From an implementation perspective, IMU drivers typically use triggers to enable hardware interrupts, print timestamps in the hardware interrupt top half, collect sensor data in the bottom half/process context, and then push the data to the buffer. The entire process is shown in the figure below.



1.3 Driver Adaptation Process for IIO Framework

This section mainly provides development guidance for IMU drivers. When adapting a new IMU chip, you can proceed in the following 6 steps.



1.3.1 DTS & Kconfig

The kernel completes bus address, pin multiplexing, and interrupt configuration through DTS. All parameters need to follow the specific hardware circuit design.

1.3.1.1 Bus Configuration

IMU communicates with SoC via I2C or SPI. Taking I2C as an example:

```
&i2c4 {
    status = "okay";
    clock-frequency = <400000>;         // Bus frequency, can be configured up to chip
maximum
    imu@68 {
        compatible = "vendor,xxx-imu";  // Must match driver's of_device_id
        reg = <0x68>;                   // I2C device address, determined by hardware
        status = "okay";
    };
};
```

- `clock-frequency`: Bus speed affects register read/write delay. It is recommended to configure to the maximum value supported by the chip. For SPI, set it in `&spiX { max-freq = <...>; }`.

1.3.1.2 Interrupt / Pin Configuration

IMU uses hardware interrupts to trigger sampling. You need to declare the interrupt pin and pinctrl multiplexing in DTS.

```
&i2c4 {
    imu@68 {
        pinctrl-names = "default";
        pinctrl-0 = <&xxx_int>;                      // Used pinctrl configuration group
        int1-gpio = <&gpio0 RK_PA4 GPIO_ACTIVE_HIGH>; // INT1 pin, default high level
        interrupt-parent = <&gpio0>;                 // GPIO controller it belongs to
```

```
        interrupts = <4 IRQ_TYPE_EDGE_FALLING>;       // PA4, falling edge trigger
    };
};

&pinctrl {
    imu {
        xxx_int: xxx-int {
            // PA4 configured as GPIO function, pull-up input
            rockchip,pins = <0 RK_PA4 RK_FUNC_GPIO &pcfg_pull_up>;
        };
    };
};
```

- `GPIO_ACTIVE_HIGH` + `IRQ_TYPE_EDGE_FALLING`: INT1 defaults to high-level output, and the interrupt signal is falling edge. Both must match the hardware schematic.
- After configuration, you can check the pin status via `/sys/kernel/debug/pinctrl/`.

1.3.1.3 Kconfig Configuration

Add `Kconfig` and `Makefile` in the driver directory to include the driver in the kernel build system:

```
config IMU_xxx
    tristate
    select IIO_BUFFER
    select IIO_TRIGGERED_BUFFER

config IMU_xxx_I2C
    tristate "xxx IMU I2C driver"
    depends on I2C
    select IMU_xxx
    select REGMAP_I2C

config IMU_xxx_SPI
    tristate "xxx IMU SPI driver"
    depends on SPI_MASTER
    select IMU_xxx
    select REGMAP_SPI
```

1.3.2 Driver Framework Setup

Create a driver directory under `drivers/iio/imu/`. Based on the underlying communication bus, optionally create `xxx_i2c.c` / `xxx_spi.c`. Declare `of_device_id` to match DTS `compatible` and register the bus driver entry. Configure `regmap_config` according to the IMU register map, so that the core layer communicates with registers through a unified API without caring about the underlying bus details.

```
static const struct of_device_id xxx_of_match[] = {
    { .compatible = "vendor,xxx-imu" }, // Match with DTS
    {}
```

```
};

static const struct regmap_config xxx_regmap_config = {
    .reg_bits     = 8,        // Register address bit width
    .val_bits     = 8,        // Register value bit width
    .max_register = 0x7F,     // Highest valid register address
};

static int xxx_i2c_probe(struct i2c_client *client)
{
    struct regmap *regmap = devm_regmap_init_i2c(client, &xxx_regmap_config);
    return xxx_core_probe(regmap, client->irq, client->name);
}

static struct i2c_driver xxx_i2c_driver = {
    .driver = {
        .name           = "xxx-imu",
        .of_match_table = xxx_of_match,
    },
    .probe = xxx_i2c_probe,
    .remove = xxx_i2c_remove,
};
module_i2c_driver(xxx_i2c_driver);
```

The SPI bus implementation is similar. The device tree match table and regmap configuration are the same as I2C.

```
static int xxx_spi_probe(struct spi_device *spi)
{
    const struct spi_device_id *id = spi_get_device_id(spi);
    struct regmap *regmap = devm_regmap_init_spi(spi, &xxx_regmap_config);
    return xxx_core_probe(regmap, spi->irq, id->name);
}

static struct spi_driver xxx_spi_driver = {
    .driver = {
        .name           = "xxx-imu",
        .of_match_table = xxx_of_match,
    },
    .probe = xxx_spi_probe,
    .remove = xxx_spi_remove,
};
module_spi_driver(xxx_spi_driver);
```

Implement core `probe` in `xxx_core.c` to allocate and register the IIO device:

```
int xxx_core_probe(struct regmap *regmap, int irq, const char *name)
{
    // IIO device initialization
    struct iio_dev *iio_dev = devm_iio_device_alloc(dev, sizeof(*data));
    iio_dev->name  = name;
    iio_dev->modes = INDIO_BUFFER_TRIGGERED;

    return devm_iio_device_register(dev, iio_dev);
}
```

1.3.3 Define IIO Device Channels and Attributes

Declare IMU sensor channels and timestamp channels in `xxx_core.c`. These typically include a three-axis accelerometer and three-axis gyroscope.

```
#define IIO_CHANNEL(_type, _axis, _index) {                \
    .type = _type,                                         \
    .modified = 1,                                         \
    .channel2 = _axis,                                     \
    .info_mask_separate = BIT(IIO_CHAN_INFO_RAW),          \
    .info_mask_shared_by_type = BIT(IIO_CHAN_INFO_SCALE) |  \
        BIT(IIO_CHAN_INFO_SAMP_FREQ),                      \
    .scan_index = _index,                                  \
    .scan_type = {                                         \
        .sign       = 's',                                 \
        .realbits   = 16,                                  \
        .storagebits = 16,                                 \
        .endianness  = IIO_BE,                             \
    },                                                     \
}

static const struct iio_chan_spec xxx_channels[] = {
    // Three-axis accelerometer
    IIO_CHANNEL(IIO_ACCEL,    IIO_MOD_X, 0),
    IIO_CHANNEL(IIO_ACCEL,    IIO_MOD_Y, 1),
    IIO_CHANNEL(IIO_ACCEL,    IIO_MOD_Z, 2),
    // Three-axis gyroscope
    IIO_CHANNEL(IIO_ANGL_VEL, IIO_MOD_X, 3),
    IIO_CHANNEL(IIO_ANGL_VEL, IIO_MOD_Y, 4),
    IIO_CHANNEL(IIO_ANGL_VEL, IIO_MOD_Z, 5),
    // Temperature
    IIO_CHANNEL(IIO_TEMP,     IIO_NO_MOD, 6),
    IIO_CHAN_SOFT_TIMESTAMP(7),
};
```

Declare IIO device attributes in `xxx_core.c`. Read/write functions can use the already encapsulated regmap for bus communication.

```
static int xxx_read_raw(struct iio_dev *indio_dev,
                        struct iio_chan_spec const *chan,
```

```
                        int *val, int *val2, long mask);
static int xxx_write_raw(struct iio_dev *indio_dev,
                         struct iio_chan_spec const *chan,
                         int val, int val2, long mask);
static const struct attribute_group xxx_attrs_group = {
    .attrs = xxx_attrs,
};

static const struct iio_info xxx_info = {
    .attrs = &xxx_attrs_group,
    .read_raw  = xxx_read_raw,
    .write_raw = xxx_write_raw,
};
```

Mount in `xxx_core_probe`:

```
int xxx_core_probe(struct regmap *regmap, int irq, const char *name)
{
    iio_dev->channels  = xxx_channels;
    iio_dev->info  = &xxx_info;
}
```

1.3.4 Chip Initialization Implementation

Driver developers need to implement this based on the chip manufacturer's FS/Datasheet. Steps include but are not limited to sensor reset, initializing gyroscope, accelerometer and other sensors, initializing clock, power and interrupt and other hardware resources.

1.3.5 Configure IIO Trigger

```
static int xxx_set_trigger_state(struct iio_trigger *trig, bool state)
{
    // Specific implementation: Enable Data Ready / Watermark interrupt
}

static const struct iio_trigger_ops xxx_trigger_ops = {
    .set_trigger_state = xxx_set_trigger_state,
};

// Register IIO trigger
int xxx_core_probe(struct regmap *regmap, int irq, const char *name)
{
    trig = devm_iio_trigger_alloc(dev, "%s-dev%d",
                                  iio_dev->name, iio_device_id(iio_dev));
    trig->ops = &xxx_trigger_ops;
    iio_trigger_set_drvdata(trig, iio_dev);
    devm_iio_trigger_register(dev, trig);
}
```

1.3.6 Configure Triggered-Buffer

As described in triggered-buffer configuration, the trigger is responsible for enabling hardware interrupts, while interrupt handling and sensor signal acquisition & push to buffer need to be implemented. This section describes how to design ISR and how to initialize and configure the buffer.

The buffer and interrupt configuration is as follows. It is recommended to use `devm_request_irq` to request hardware interrupts and trigger sampling via `iio_trigger_generic_data_rdy_poll`. Sampling is divided into two steps: first, enter `iio_pollfunc_store_time` to record the timestamp, and second, collect sensor data in `xxx_handler` and push to the buffer.

```
int xxx_core_probe(struct regmap *regmap, int irq, const char *name)
{
    devm_iio_triggered_buffer_setup(dev, indio_dev,
                                    iio_pollfunc_store_time, xxx_handler, NULL);
    devm_request_irq(&iio_dev->dev, irq,
                     &iio_trigger_generic_data_rdy_poll,
                     irq_type, name, trig);
}
```

Which sensor information to collect is determined by the user. Which addresses to read depends on the IMU's own design configuration. Only the framework is listed here.

```
static irqreturn_t xxx_handler(int irq, void *p)
{
    // iio_pollfunc_store_time records the timestamp in the top-half, the bottom-half reads
it directly
    struct iio_poll_func *pf = p;
    s64 data_timestamp = pf->timestamp;

    // Collect data to sensor_data
    u8 sensor_data[13];
    unsigned int val;
    regmap_bulk_read(regmap, 0x01, sensor_data, 12);    // Start from 0x01, continuously read
12 regs
    regmap_read(regmap, 0x3A, &val);
    sensor_data[12] = (u8)val;                          // Read 0x3A register

    // Push to buffer
    iio_push_to_buffers_with_timestamp(iio_dev, sensor_data, data_timestamp);
    return IRQ_HANDLED;
}
```

2. IMU User Guide

This chapter introduces methods for using sensors through the Linux IIO interface, including sysfs parameter configuration, interrupt acquisition enable, and data reading.

2.1 IIO sysfs Configuration

The IIO driver exposes sensor parameters through sysfs. This section describes methods for user space to configure IIO sysfs files to modify sensor parameters and enable triggers.

2.1.1 Sampling Frequency

The sampling frequency (ODR) is read and written through the `in_{type}_sampling_frequency` file. The optional value list is provided by the `_available` suffix file.

```
# View supported sampling frequencies (unit: Hz)
cat /sys/bus/iio/devices/iio:deviceX/in_accel_sampling_frequency_available

# Set sampling frequency
echo 400 > /sys/bus/iio/devices/iio:deviceX/in_accel_sampling_frequency

# Writing a value not in the list will return -EINVAL
echo 9 > /sys/bus/iio/devices/iio:deviceX/in_accel_sampling_frequency
-bash: echo: write error: Invalid argument
```

The ODR files for accelerometer and gyroscope are independent, namely `in_accel_sampling_frequency` and `in_anglvel_sampling_frequency`.

2.1.2 Range and Precision

The range is set through the `in_{type}_scale` file, in units of SI physical quantity per LSB. Precision is negatively correlated with range:

$$scale*{accel} = *frac{range*{g} \times 2 \times 9.8}{2^{bits}}\quad(m/s^2/LSB)$$

$$scale*{gyro} = *frac{range*{dps} \times 2 \times \pi}{2^{bits} \times 180}\quad(rad/s/LSB)$$

```
# View accelerometer optional precision (unit: m/s²/LSB)
cat /sys/bus/iio/devices/iio:deviceX/in_accel_scale_available

# Set accelerometer range
echo 0.000598 > /sys/bus/iio/devices/iio:deviceX/in_accel_scale

# View gyroscope optional precision (unit: rad/s/LSB)
cat /sys/bus/iio/devices/iio:deviceX/in_anglvel_scale_available

# Set gyroscope range
echo 0.000133231 > /sys/bus/iio/devices/iio:deviceX/in_anglvel_scale
```

Range Selection Suggestions:

- **Large range** - Can capture fast/vigorous motion without saturation, suitable for **drones, action cameras, and handheld gimbals** and other scenarios with intense motion.
- **Small range** - High resolution, small quantization noise, but easily truncated during fast motion, suitable for **robot chassis, AGV, and static attitude estimation** and other scenarios with gentle motion.

2.1.3 Low-Pass Filter

The LPF cutoff frequency is set through the `in_{type}_lpf_bw` file (some drivers use `filter_low_pass_3db_frequency`). Writing 0 means bypass.

```
# View optional cutoff frequencies (unit: Hz)
cat /sys/bus/iio/devices/iio:deviceX/in_accel_lpf_bw_available

# Set to 180 Hz (block signals higher than 180 Hz)
echo 180 > /sys/bus/iio/devices/iio:deviceX/in_accel_lpf_bw

# Disable filter (bypass)
echo 0 > /sys/bus/iio/devices/iio:deviceX/in_accel_lpf_bw
```

LPF bandwidth should be coordinated with ODR and range: in high-dynamic scenarios, increase the sampling rate and relax the LPF; in low-dynamic scenarios, decrease the sampling rate and narrow the LPF to suppress high-frequency noise.

2.1.4 Interrupt Enable

The IIO driver implements interrupt-driven acquisition through the trigger + buffer mechanism. The enable steps are independent of specific chips.

```
# Find trigger id and name
ls /sys/bus/iio/devices/ | grep trigger
# triggerX

cat /sys/bus/iio/devices/triggerX/name
# <trigger_name>
```

```
# Find corresponding IIO device id
cat /sys/bus/iio/devices/iio:deviceX/name

cd /sys/bus/iio/devices/iio:deviceX

# Bind trigger
echo "<trigger_name>" > trigger/current_trigger

# Enable channels to collect
echo 1 > ./scan_elements/in_accel_x_en
echo 1 > ./scan_elements/in_accel_y_en
echo 1 > ./scan_elements/in_accel_z_en
echo 1 > ./scan_elements/in_anglvel_x_en
echo 1 > ./scan_elements/in_anglvel_y_en
echo 1 > ./scan_elements/in_anglvel_z_en
echo 1 > ./scan_elements/in_temp_x_en
echo 1 > ./scan_elements/in_timestamp_en

# Configure kfifo buffer
echo 128 > ./buffer/length
echo 1 > ./buffer/watermark

# Enable interrupt-triggered acquisition
echo 1 > ./buffer/enable

# Disable acquisition
echo 0 > ./buffer/enable
```

- `scan_elements/in_*_en`: Enable as needed. Channels not enabled do not occupy data frame bandwidth.
- `buffer/length`: Number of frames in the kfifo circular buffer. It is recommended to set to 0.1 to 0.5 times the ODR.
- `buffer/watermark`: After accumulating to this number of frames, `poll()` is awakened. Setting to 1 is equivalent to awakening once per frame.
- After enabling, the driver calls `trigger_set_state(true)` to enable hardware interrupts; when closing, `trigger_set_state(false)` is called to disable interrupts. Data is read via `read()` from `/dev/iio:deviceX`.

2.2 Data Reading and Processing

This section describes methods for reading accelerometer/gyroscope data through the Linux IIO interface.

2.2.1 IIO sysfs Read

```
# Get three-axis accelerometer sensor data
cat /sys/bus/iio/devices/iio\:device1/in_accel_x_raw
cat /sys/bus/iio/devices/iio\:device1/in_accel_y_raw
cat /sys/bus/iio/devices/iio\:device1/in_accel_z_raw

# Get three-axis gyroscope sensor data
cat /sys/bus/iio/devices/iio\:device1/in_anglvel_x_raw
cat /sys/bus/iio/devices/iio\:device1/in_anglvel_y_raw
cat /sys/bus/iio/devices/iio\:device1/in_anglvel_z_raw

# Get temperature sensor data
cat /sys/bus/iio/devices/iio\:device1/in_temp_x_raw
```

To obtain raw accelerometer/gyroscope data, you need to multiply by the precision unit. For example:

```
cat /sys/bus/iio/devices/iio\:device1/in_accel_scale
0.000598

cat /sys/bus/iio/devices/iio\:device1/in_accel_z_raw
16412

# Calculate Z-axis acceleration
16412×0.000598 = 9.814376 m/s²
```

> **Note:** The IIO framework outputs gyroscope angular velocity in **rad/s (radians per second)**, not degrees per second (°/s). Interpreting the data as °/s will cause values to be inflated by approximately 57× (i.e., 180/π), leading to abnormal attitude estimation results. Always verify the unit before use.

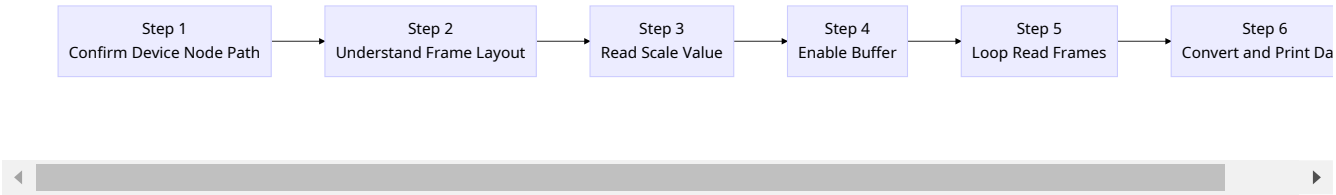
Temperature sensor data needs to be converted according to the register value. The conversion formula depends on the temperature sensor quantity in the packet:

`8-bit quantity (FIFO):` $$Celsius = \frac{TEMP_{DATA}}{2} + 25$$

`16-bit quantity (DRDY):` $$Celsius = \frac{TEMP_{DATA}}{128} + 25$$

2.2.2 IIO Buffer Read Process

User space reads IMU data from `/dev/iio:deviceX` through `read` / `poll` system calls. The overall process is divided into 6 steps.



2.2.2.1 Confirm Device Node Path

The IIO framework creates two types of access entry points for each registered IMU device:

- **Character device** `/dev/iio:deviceX`: Used for `read()` batch reading of sampling frames
- **sysfs node** `/sys/bus/iio/devices/iio:deviceX/`: Used for reading sensor parameters (scale, ODR, etc.)

```
# View how many IIO devices are currently in the system
ls /sys/bus/iio/devices/

# View device name (corresponds to iio_dev->name in the driver)
cat /sys/bus/iio/devices/iio:device1/name
```

Driver developers need to ensure that the `compatible` in DTS matches the driver, so the device node will appear after system startup.

2.2.2.2 Understand Frame Layout

Data frames read from `/dev/iio:deviceX` consist of **scan elements (channel data) + timestamp**. The specific layout is determined by `iio_chan_spec.scan_type` at the driver layer. For example, the frame format defined by the ICM42670 driver is as follows:

Offset	Field	Type	Description
0	ax	s16 BE	Accelerometer X-axis raw value
2	ay	s16 BE	Accelerometer Y-axis raw value
4	az	s16 BE	Accelerometer Z-axis raw value
6	gx	s16 BE	Gyroscope X-axis raw value
8	gy	s16 BE	Gyroscope Y-axis raw value
10	gz	s16 BE	Gyroscope Z-axis raw value
12	temp	s8/s16	Temperature raw value
16	timestamp	s64 LE	Timestamp (ns)

```
struct imu_frame {
    int16_t  ax, ay, az;   /* accel raw, BE */
    int16_t  gx, gy, gz;   /* gyro  raw, BE */
    int16_t  temp;         /* temp  raw, 16-bit */
    uint8_t  pad[2];       /* padding to 8B margin */
    int64_t  timestamp;    /* ns, LE */
} __attribute__((packed));
```

2.2.2.3 Read Scale Value

`read_raw` returns the raw sensor register value, which needs to be multiplied by `scale` to get the physical quantity. Scale is read through sysfs files:

```
float accel_scale = read_sysfs_float("/sys/bus/iio/devices/iio:device1/in_accel_scale");
float gyro_scale  = read_sysfs_float("/sys/bus/iio/devices/iio:device1/in_anglvel_scale");
```

These two files correspond to `info_mask_shared_by_type = BIT(IIO_CHAN_INFO_SCALE)` defined in the driver. If the file does not exist or reading fails, the default value `0.0f` is returned, and all data will be 0.

2.2.2.4 Enable Buffer

Before reading data in user space, you need to enable the buffer through sysfs. For configuration, refer to Interrupt Enable. After enabling the buffer, the driver's `xxx_handler` starts receiving interrupts, and the collected data is pushed to `kfifo`, so that user space `read()` has data to read.

2.2.2.5 Loop Read Frames

Open the character device and collect data in a blocking `read()` loop:

```
int fd = open("/dev/iio:device1", O_RDONLY);
if (fd < 0) {
    perror("open");
    return 1;
}

struct imu_frame frame;
while (running) {
    ssize_t n = read(fd, &frame, sizeof(frame));
    if (n < 0) { perror("read"); break; }
    if ((size_t)n != sizeof(frame)) {
        fprintf(stderr, "short read: %zd (expected %zu)\n", n, sizeof(frame));
        continue;
    }
    // Process frame ...
}
```

`read()` returns one complete frame of data (24 bytes) each time. If the driver's buffer length configuration is greater than 1, `read()` may return multiple frames at once. In this case, `n` will be an integer multiple of `sizeof(frame)`.

2.2.2.6 Convert and Print Data

The `scan_type.endianness` on the driver side determines the storage byte order of the data. If it is big-endian, user space needs to convert to host byte order through `ntohs()` after reading. Timestamp is fixed as little-endian. Convert raw values to physical quantities and print:

```
#include <arpa/inet.h>   // ntohs

// Big-endian → host byte order
int16_t ax = ntohs(frame.ax);
int16_t ay = ntohs(frame.ay);
int16_t az = ntohs(frame.az);

// Raw value × scale = physical quantity
float ax_phys = ax * accel_scale;   // m/s²
float gx_phys = gx * gyro_scale;    // rad/s

// Calculate sampling interval
float dt_ms = (float)(frame.timestamp - prev_ts) / 1e6f;
prev_ts = frame.timestamp;

printf("a(m/s2) %6.3f %6.3f %6.3f | g(rad/s) %7.4f %7.4f %7.4f\n",
       ax_phys, ay_phys, az_phys, gx_phys, gy_phys, gz_phys);
```

The reference Demo code is as follows (developed using ICM 42670 as the template. Compile with `gcc -o iio_icm42670_read iio_icm42670_read.c` for DRDY interrupt sampling. To read FIFO watermark interrupt sampling, add `-DFIFO_MODE` during compilation):

```
// iio_icm42670_read.c

/* Execution:
    chmod +x ./iio_icm42670_read
    ./iio_icm42670_read
*/

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <arpa/inet.h>   /* ntohs: big-endian conversion */
#include <signal.h>

#define IIO_DEV    "/dev/iio:device1"
#define IIO_SYSFS  "/sys/bus/iio/devices/iio:device1"

/* ----------------------------------------------------------------
 * Frame layout (accel/gyro are exactly the same in both modes, only temp field size differs)
 *
 * FIFO mode (-DFIFO_MODE, DTS has enable-fifo):
 *   offset  0: ax  s16 BE  2B
 *   offset  2: ay  s16 BE  2B
 *   offset  4: az  s16 BE  2B
 *   offset  6: gx  s16 BE  2B
 *   offset  8: gy  s16 BE  2B
 *   offset 10: gz  s16 BE  2B
 *   offset 12: temp s8     1B  ← 8-bit, precision 0.5°C/LSB
 *   offset 13: pad        3B
 *   offset 16: timestamp s64  8B (LE, ns)
 *   Total 24 B
 *
 * DRDY mode (default, DTS has no enable-fifo):
 *   offset  0~11: same as above
 *   offset 12: temp s16   2B  ← 16-bit, precision 1/128°C/LSB
 *   offset 14: pad        2B
 *   offset 16: timestamp s64  8B (LE, ns)
 *   Total 24 B
 * ---------------------------------------------------------------- */

#ifdef FIFO_MODE
struct imu_frame {
    int16_t  ax, ay, az;   /* accel raw, BE */
    int16_t  gx, gy, gz;   /* gyro  raw, BE */
    int8_t   temp;         /* temp  raw, 8-bit */
    uint8_t  pad[3];
```

```
    int64_t  timestamp;    /* ns, LE */
} __attribute__((packed));

/* T(°C) = (raw + 50) * 0.5 */
static inline float conv_temp(int8_t raw)
{
    return (raw + 50.0f) * 0.5f;
}
#else
struct imu_frame {
    int16_t  ax, ay, az;   /* accel raw, BE */
    int16_t  gx, gy, gz;   /* gyro  raw, BE */
    int16_t  temp;         /* temp  raw, 16-bit */
    uint8_t  pad[2];
    int64_t  timestamp;    /* ns, LE */
} __attribute__((packed));

/* T(°C) = (raw + 3200) / 128.0 */
static inline float conv_temp(int16_t raw)
{
    return (raw + 3200.0f) / 128.0f;
}
#endif

static int running = 1;
static void on_sigint(int s) { (void)s; running = 0; }

static float read_sysfs_float(const char *path)
{
    FILE *f = fopen(path, "r");
    float v = 0.0f;
    if (f) { fscanf(f, "%f", &v); fclose(f); }
    else   { perror(path); }
    return v;
}

/* big-endian int16 → host int16 */
static inline int16_t be16_to_host(int16_t v)
{
    return (int16_t)ntohs((uint16_t)v);
}

int main(void)
{
    signal(SIGINT, on_sigint);

#ifdef FIFO_MODE
    printf("Mode: FIFO  (temp 8-bit, 0.5 deg/LSB)\n");
#else
    printf("Mode: DRDY  (temp 16-bit, 1/128 deg/LSB)\n");
#endif

    float accel_scale = read_sysfs_float(IIO_SYSFS "/in_accel_scale");
```

```
    float gyro_scale  = read_sysfs_float(IIO_SYSFS "/in_anglvel_scale");

    printf("accel_scale = %.9f m/s2/LSB\n", accel_scale);
    printf("gyro_scale  = %.9f rad/s/LSB\n", gyro_scale);
    printf("frame size  = %zu bytes\n", sizeof(struct imu_frame));
    printf("Ctrl-C to stop\n\n");

    int fd = open(IIO_DEV, O_RDONLY);
    if (fd < 0) {
        perror("open " IIO_DEV);
        fprintf(stderr, "Please enable buffer first: echo 1 > " IIO_SYSFS
"/buffer/enable\n");
        return 1;
    }

    struct imu_frame frame;
    int count = 0;
    int64_t prev_ts = 0;

    while (running) {
        ssize_t n = read(fd, &frame, sizeof(frame));
        if (n < 0) { perror("read"); break; }
        if ((size_t)n != sizeof(frame)) {
            fprintf(stderr, "short read: %zd (expected %zu)\n", n, sizeof(frame));
            continue;
        }

        float ax = be16_to_host(frame.ax) * accel_scale;
        float ay = be16_to_host(frame.ay) * accel_scale;
        float az = be16_to_host(frame.az) * accel_scale;
        float gx = be16_to_host(frame.gx) * gyro_scale;
        float gy = be16_to_host(frame.gy) * gyro_scale;
        float gz = be16_to_host(frame.gz) * gyro_scale;
        float tc = conv_temp(frame.temp);

        float dt_ms = prev_ts ? (float)(frame.timestamp - prev_ts) / 1e6f : 0.0f;
        prev_ts = frame.timestamp;

        printf("[%5d] ts=%lld dt=%.2fms | "
               "a(m/s2) %6.3f %6.3f %6.3f | "
               "g(rad/s) %7.4f %7.4f %7.4f | "
               "T=%.1fC\n",
               count++,
               (long long)frame.timestamp,
               dt_ms,
               ax, ay, az,
               gx, gy, gz,
               tc);
    }

    close(fd);
    printf("\nTotal frames: %d\n", count);
```

```
    return 0;
}
```

3. Data Analysis and Calibration

This chapter introduces IMU data quality assessment and calibration methods.

3.1 Data Calibration and Calibration

Both EIS and SLAM scenarios require calibration. EIS mainly focuses on consistency calibration between IMU and image rotation directions. For EIS calibration methods, refer to *Rockchip_Developer_Guide_EIS_CN.pdf* (located at `docs/cn/Socs/RV1126B/RKIPC/`).

For SLAM use cases, you need to calibrate the extrinsic parameters (relative pose) between the IMU and camera, as well as the IMU noise parameters. The calibration results are used to fill in the algorithm configuration file. The following sections mainly describe the SLAM extrinsic and IMU noise calibration flow, using **kalibr** as the recommended tool.

3.1.1 Understand the Calibration Parameters

SLAM algorithms (e.g., ORB-SLAM3) require the following parameters to fuse IMU data:

Parameter	Description	Unit
`T_b_c`	Transformation matrix from camera frame to IMU frame (4×4)	—
`NoiseGyro`	Gyroscope random noise density	rad/s/√Hz
`NoiseAcc`	Accelerometer random noise density	$m/s^2/\sqrt{Hz}$
`GyroWalk`	Gyroscope bias random walk	$rad/s^2/\sqrt{Hz}$
`AccWalk`	Accelerometer bias random walk	$m/s^3/\sqrt{Hz}$
`Frequency`	IMU data publishing frequency	Hz

`T_b_c` must be obtained through data collection and calibration. Noise parameters can be found in the Allan variance curves in the chip datasheet, or estimated together by kalibr's `kalibr_calibrate_imu_camera` command.

3.1.2 Install the Calibration Tool

```
# Clone the kalibr ROS2 source
git clone https://github.com/yuhang0131/kalibr_ros2.git
cd kalibr_ros2
```

```
# Install system dependencies
sudo apt install -y \
  libeigen3-dev libboost-all-dev libopencv-dev \
  libsuitesparse-dev libtbb-dev \
  python3-numpy python3-matplotlib python3-scipy python3-opencv

# Install Python dependencies
pip install python-igraph

# Build (if a missing dependency is reported, source and re-run)
./build_workspace.sh
source install/setup.bash
```

3.1.3 Collect Calibration Data

Before recording, confirm that the camera and IMU timestamps are synchronized (hardware sync or software time alignment); otherwise the estimated time-shift will be unreliable.

Checkerboard target recording should follow the requirements below.

Checkerboard Requirements

- Use a flat, rigid board. Do not use curled or reflective surfaces.
- Checker size and corner count (internal corners) must be known and accurate, and must match the `target.yaml` parameters.
- Ensure clear black-white contrast with clean and sharp edges.

Motion Requirements

- Keep the camera and IMU **rigidly connected** throughout the session, and record approximately 120 s of bag data while facing a checkerboard target.
- During recording, execute the following translation + rotation + figure-8 sequence in order:

1. Stay stationary 1–2 m from the target for 10 s (to estimate initial bias)
2. Translate forward/backward, left/right, up/down for 20 s
3. Rotate in pitch, yaw, and roll for 20 s
4. Figure-8 trajectory + random motion for 30 s, keep motion continuous and excitation sufficient
5. Vary distance and angle while observing the target for 30 s
6. Return to the starting position and stay still for 10 s

Image Capture Requirements

- Do not let the checkerboard appear too small. It is recommended to occupy at least 1/4 to 1/2 of the image.
- Do not keep the checkerboard in one region for too long. Cover center + corners + edges, with varied distance and tilt angles (pitch/yaw/roll).
- Avoid strong reflections, flickering light sources, motion blur, and severe overexposure.

```
# Record RGB image and IMU data simultaneously, ~120 s
timeout 120s ros2 bag record -o imu_cam_calibrate \
  /your_cam_topic/color/image_rgb \
  /your_imu_topic/data
```

3.1.4 Create Configuration Files

camchain.yaml (camera intrinsics from factory calibration)

```
cam0:
  camera_model: pinhole
  intrinsics: [fx, fy, cx, cy]        # fill in actual intrinsics
  distortion_model: radtan
  distortion_coeffs: [k1, k2, p1, p2] # use [0,0,0,0] if factory-corrected
  resolution: [width, height]
  rostopic: /your_cam_topic/color/image_rgb
```

imu.yaml (IMU noise parameters from datasheet; use empirical values below if uncertain)

```
accelerometer_noise_density: 0.006      # m/s^2/sqrt(Hz)
accelerometer_random_walk:   0.0002     # m/s^3/sqrt(Hz)
gyroscope_noise_density:     0.0004     # rad/s/sqrt(Hz)
gyroscope_random_walk:       4.0e-06    # rad/s^2/sqrt(Hz)
update_rate: 200.0                      # Hz, must match actual IMU ODR
rostopic: /your_imu_topic/data
```

target.yaml (checkerboard target parameters, fill in according to the physical target)

```
target_type: 'checkerboard'
targetCols: 8            # number of internal corner columns
targetRows: 11           # number of internal corner rows
rowSpacingMeters: 0.05   # row corner spacing (m)
colSpacingMeters: 0.05   # column corner spacing (m)
```

3.1.5 Run Calibration

```
cd kalibr_ros2 && source install/local_setup.bash

ros2 run kalibr_imu_camera kalibr_calibrate_imu_camera \
  --bag  imu_cam_calibrate \
  --cams camchain.yaml \
  --imu  imu.yaml \
  --target target.yaml \
  --show-extraction
```

After calibration, kalibr outputs `*-results-imucam.txt` and `*-camchain.yaml` in the current directory, containing:

- `T_ci / T_ic`: extrinsic matrix from camera to IMU / from IMU to camera
- `timeshift_cam0_to_imu0`: time offset of camera relative to IMU (s)
- Fitted estimates of noise parameters

```
Calibration results
====================
Normalized Residuals
----------------------------
Reprojection error (cam0):     mean 0.09968181195443296, median 0.0903287683591986, std: 0.060634278731619765
Gyroscope error (imu0):        mean 0.3250498564274567, median 0.25135580409887376, std: 0.8993709899513046
Accelerometer error (imu0):    mean 0.292109487199641, median 0.1980429974587293, std: 0.9634169383777025

Residuals
----------------------------
Reprojection error (cam0) [px]:     mean 0.09968181195443296, median 0.0903287683591986, std:
0.060634278731619765
Gyroscope error (imu0) [rad/s]:     mean 0.0018387596616285466, median 0.0014218831485512883, std:
0.0050876106063762055
Accelerometer error (imu0) [m/s^2]: mean 0.024786311909734942, median 0.01680450557634932, std:
0.08174863802842267

Transformation (cam0):
-----------------------
T_ci:  (imu0 to cam0):
[[-0.99974638  0.0038015   0.02219753 -0.00044106]
 [-0.00412274 -0.99988717 -0.01444453 -0.0080838 ]
 [ 0.02214011 -0.01453238  0.99964925 -0.00744897]
 [ 0.         0.         0.         1.        ]]

T_ic:  (cam0 to imu0):
[[-0.99974638 -0.00412274  0.02214011 -0.00030935]
 [ 0.0038015  -0.99988717 -0.01453238 -0.00818946]
 [ 0.02219753 -0.01444453  0.99964925  0.00733938]
 [ 0.         0.         0.         1.        ]]

timeshift cam0 to imu0: [s] (t_imu = t_cam + shift)
-0.001701628961663 59

Gravity vector in target coords: [m/s^2]
[ 9.66151343  0.20447352 -1.66786444]
```

3.1.6 Write Calibration Results into the Algorithm Configuration

Fill the outputs into the ORB-SLAM3 yaml configuration file:

```
IMU.T_b_c1: !!opencv-matrix
   rows: 4
   cols: 4
   dt: f
   data: [ r00, r01, r02, t0,
           r10, r11, r12, t1,
           r20, r21, r22, t2,
           0.,  0.,  0.,  1. ]   # from T_imu_cam

IMU.NoiseGyro:  0.0004    # from calibration result or datasheet
IMU.NoiseAcc:   0.006
IMU.GyroWalk:   4.0e-06
IMU.AccWalk:    0.0002
IMU.Frequency:  200       # must match update_rate in imu.yaml
```

3.2 Determining Data Normalcy

This section describes how to determine whether IMU data is normal or not. Use the `visualize_imu.py` script to parse IMU data and judge the accuracy of input data from 10 dimensions.

The input IMU data must follow an 8-column format per line, with a fixed column order:

```
accel_x  accel_y  accel_z  angle_x  angle_y  angle_z  temp  timestamp
```

Where acceleration unit is m/s², angular velocity unit is rad/s, temperature is the raw register value or degrees Celsius (determined by `TEMP_MODE` at the top of the script), and timestamp unit is nanoseconds (ns).

3.2.1 Data Acquisition

On the development board, loop-read through IIO sysfs and append results to a log file. A sample script is as follows:

```
#!/bin/sh

DEV=/sys/bus/iio/devices/iio:device1
OUT=./imu.log

# Read scale once, subsequent direct conversion to physical quantity
A_SCALE=$(cat $DEV/in_accel_scale)
G_SCALE=$(cat $DEV/in_anglvel_scale)

: > $OUT
for i in $(seq 1 2000); do
    AX=$(cat $DEV/in_accel_x_raw)
    AY=$(cat $DEV/in_accel_y_raw)
    AZ=$(cat $DEV/in_accel_z_raw)
    GX=$(cat $DEV/in_anglvel_x_raw)
    GY=$(cat $DEV/in_anglvel_y_raw)
    GZ=$(cat $DEV/in_anglvel_z_raw)
    T=$(cat $DEV/in_temp_x_raw)
    TS=$(date +%s%N)
    awk -v ax=$AX -v ay=$AY -v az=$AZ \
        -v gx=$GX -v gy=$GY -v gz=$GZ \
        -v t=$T -v ts=$TS \
        -v as=$A_SCALE -v gs=$G_SCALE \
        'BEGIN{printf "%.6f %.6f %.6f %.6f %.6f %.6f %d %s\n",
               ax*as, ay*as, az*as, gx*gs, gy*gs, gz*gs, t, ts}' >> $OUT
done
```

In actual use, it is recommended to enable the IIO trigger and collect through IIO buffer to obtain stable high-frequency data. The above shell script polling is only for quick verification.

3.2.2 Data Check

The data check script is stored in the cloud and can be downloaded via this link. Copy the IMU sampling log `imu.log` to PC and execute:

```
python visualize_imu.py imu.log
```

The script will output `imu.log.png` in the same directory. The figure contains 11 subplots: accelerometer/angle raw curves and integration, temperature, timestamp interval and jitter, distribution histogram, anomaly annotation, and statistical summary. The script has a built-in `AnomalyDetector` that checks data from the following dimensions. If the count of any item is greater than 0, it is considered an anomaly:

Detection Item	Criterion	Common Cause
`duplicate_timestamp`	Adjacent timestamps are exactly the same	Acquisition thread preempted, sysfs read rate too slow
`timestamp_jump`	Adjacent interval appears < -1 ms backward	System clock modified, multi-thread disorderly write
`timestamp_gap`	Adjacent interval deviates from mean by more than `TIMESTAMP_JITTER_THRESHOLD` (default 100 μs)	Interrupt delay, I2C bus blocking
`accel_out_of_range`	Accelerometer out of range	Range configured too small causing saturation, or mechanical shock
`angle_out_of_range`	Gyroscope out of range	Gyroscope range too small, data parsing error
`temp_out_of_range`	Temperature sensor out of range	Temperature register not correctly converted
`accel_outlier` / `angle_outlier`	Z-score > 3 or exceeds Q3 ± 3·IQR	Single-point jitter, bus error
`gyro_rate_outlier`	Adjacent angular velocity change > 5 rad/s	Sampling interval unstable, data misalignment
`stuck_value`	≥5 consecutive samples are exactly the same	Driver not updating, FIFO stuck
`invalid_data`	`NaN` / `Inf` appears	Raw register read failure

The terminal also prints a statistical summary. Panel 11 in the figure also lists key anomaly counts and `mean ± std` for each channel. **Process for determining data normalcy:**

- Check if the total sample count and actual sampling rate match the configured ODR. If the deviation is significant, first investigate the acquisition process.
- If `duplicate_timestamp` / `timestamp_gap` is non-zero, it indicates jitter in the acquisition link. Priority should be given to resolving this rather than suspecting the sensor.

- If `accel_out_of_range` or `angle_out_of_range` is non-zero, check whether the current range matches the motion amplitude. Increase the range if necessary (refer to Range and Precision Configuration).
- If `stuck_value` / `invalid_data` is non-zero, it is usually a driver or bus exception. Further locate with `dmesg`.
- In a stationary state, the three-axis accelerometer magnitude should be close to $9.8\,\text{m/s}^2$, and the three-axis gyroscope mean should be close to 0. If the deviation is too large, execute Data Calibration and Calibration.

4. FAQ

This chapter mainly describes common problems and corresponding troubleshooting methods during IMU driver debugging.

4.1 PCB Layout Precautions

- Decoupling capacitors (100 nF + 10 μF) should be placed as close as possible to the power pin to reduce power domain EMI.
- Keep away from high-frequency circuits (Wi-Fi antennas, switching power supplies) and mechanical vibration sources (fans, motors).
- If the resonance frequency of mounting posts/adhesive layers is within the ODR frequency band, a fixed peak will appear in the spectrum.
- For more complete layout specifications, refer to TDK's *MEMS Products PCB Design, Mounting, and Handling Guidelines* and Bosch's *BMI08x Design Guide*.

4.2 Is Zero Bias and Temperature Drift Factory Calibration Needed

- **Consumer/EIS scenarios**: Real-time compensation at the application layer (stationary average) is sufficient. No factory calibration is needed. However, ensure that the driver `calibbias` attribute is exposed.
- **SLAM/Navigation scenarios**: Factory six-face zero-bias calibration is recommended, and linear temperature compensation coefficients for the full temperature range (-20 ~ 60°C) are fitted, written to NVM or loaded in user space at startup.
- For specific calibration methods, refer to each chip's datasheet `Self-Test and OTP` chapter.

4.3 Effect of Built-in LPF on Data

- The lower the LPF cutoff frequency, the smaller the noise, but **phase delay is introduced**. The specific delay characteristics must be referenced from the chip datasheet.
- Algorithms sensitive to phase consistency, such as SLAM/VIO, should be aware that the built-in LPF may cause misalignment between the timestamp and actual motion. Evaluate whether to disable or adjust the LPF based on algorithm requirements.